

4CS02 – Cryptologie
Travaux Dirigés
Cryptographie à clé publique
Corrigé

Année 2024

1 Récapitulatif du chiffrement RSA

On utilise les notations habituelles du chiffrement RSA : N est un entier et p et q sont deux entiers premiers tels que $N = pq$. On note φ l'indicatrice d'Euler $\varphi = \varphi(N) = (p - 1)(q - 1)$ et e et d sont deux éléments de $\mathbb{Z}/N\mathbb{Z}^*$ tels que $ed = 1 \pmod{\varphi}$.

1. On souhaite utiliser l'algorithme de chiffrement RSA. Rappeler la valeur de la clé publique et celle de la clé secrète. Comment chiffre-t-on un message m ? Et comment déchiffre-t-on un message c ?
2. Parmi les entiers N , p , q , φ , e et d , quels sont ceux qui doivent rester secrets ? A l'aide des éléments publics, montrer que la divulgation de l'une quelconque des valeurs privées permet de retrouver toutes les autres valeurs privées.
3. Quel est l'inverse de 89 modulo φ si $\varphi = 197$?
4. On pose désormais $p = 5$ et $q = 13$. Calculer N et φ . Nous prenons $e = 5$ et $d = 29$. Soit $c = 49$ un chiffré. Retrouver le message m déchiffrant c à l'aide de la clé privée et de l'exponentiation modulaire rapide.

Réponse.

1. Pour utiliser le chiffrement RSA, on procède de la façon suivante.
 - Génération des clés : on choisit au hasard deux grands nombres premiers p et q distincts et on forme leur produit $N = p \cdot q$. On choisit au hasard un entier e premier avec $\varphi = (p - 1) \cdot (q - 1)$. Le couple (N, e) est la clé publique. La clé privée est alors l'entier d inverse de e modulo φ .
 - Chiffrement : on convertit le message en un entier $m \in [1, N - 1]$ et on calcule le chiffré $c = m^e \pmod{N}$.
 - Déchiffrement : on calcule $m' = c^d \pmod{N}$.

2. Les éléments qui doivent rester secrets sont p , q , φ et d .

Révéler p (resp. q) permet d'obtenir $q = N/p$ (resp. $p = N/q$), puis φ et donc d . Révéler φ permet d'obtenir d à partir de e .

Il reste à voir comment d permet d'obtenir le reste. Supposons donc e et d connus (ainsi que N bien sûr). Comme $e \cdot d = 1 \pmod{\varphi}$, il existe un entier k tel que $e \cdot d = 1 + k\varphi$. On sait que $\varphi = (p-1)(q-1) = (e \cdot d - 1)/k$, ce qui nous donne $(e \cdot d - 1)/k - 1 = N - p - q$ car $N = p \cdot q$. On obtient donc le système

$$\begin{cases} p + q &= N - (e \cdot d - 1)/k + 1 \\ p \cdot q &= N \end{cases}$$

Or, nous savons qu'en connaissant $p + q$ et $p \cdot q$, il est facile de retrouver p et q . Mais il nous manque encore la valeur k (seule valeur inconnue dans ce système) pour pouvoir conclure. En fait, nous n'allons pas réussir à trouver k de façon exacte tout de suite. Il va dans un premier temps nous falloir l'approximer.

Pour cela, nous allons nous appuyer sur la relation $\varphi = (e \cdot d - 1)/k$, et donc $k = (e \cdot d - 1)/\varphi$, et sur le fait qu'en pratique, p et q sont grands par rapport à 1, donc $p + q = o(p \cdot q)$. Nous allons aussi simplifier cette relation en notant que $\varphi = N - (p + q) + 1$. Ainsi, nous obtenons

$$k = \frac{e \cdot d - 1}{\varphi} = \frac{e \cdot d - 1}{N - (p + q) + 1} = \frac{e \cdot d - 1}{N(1 + o(\frac{p+q}{p \cdot q}))} = \frac{e \cdot d - 1}{N(1 + o(1))} \approx \frac{e \cdot d}{N}.$$

Donc k est proche de ed/N , il suffit d'essayer de résoudre ce système pour des valeurs proches de k , par exemple celles de l'intervalle $[e \cdot d/N - 5, e \cdot d/N + 5]$. Quand on obtient une solution (p, q) à valeurs entières, on a trouvé k et donc φ puis e .

3. Cherchons un entier $k \in [1, 196]$ tel que $89 \cdot k = 1 \pmod{197}$, c'est-à-dire deux entiers $(k, l) \in [1, 196]^2$ tels que $89 \cdot k = 1 + 197 \cdot l$. Utilisons pour cela l'algorithme d'Euclide appliqué à 197 et 89 :

$$\begin{aligned} 197 &= 89 \cdot 2 + 19 \\ 89 &= 19 \cdot 4 + 13 \\ 19 &= 13 \cdot 1 + 6 \\ 13 &= 6 \cdot 2 + 1 \end{aligned}$$

On en déduit, en remplaçant successivement chaque reste de cette suite d'égalités :

$$\begin{aligned} 1 &= 13 - 6 \cdot 2 \\ &= 13 - (19 - 13 \cdot 1) \cdot 2 = 19 \cdot (-2) + 13 \cdot 3 \\ &= 19 \cdot (-2) + (89 - 19 \cdot 4) \cdot 3 \\ &= 89 \cdot 3 + 19 \cdot (-14) \\ &= 89 \cdot 3 + (197 - 89 \cdot 2) \cdot (-14) \\ &= 197 \cdot (-14) + 89 \cdot 31 \end{aligned}$$

L'inverse de 89 modulo 197 est donc 31.

4. On a $N = 65 = 5 \cdot 13$, donc $p = 5$, $q = 13$ (ou l'inverse) et $\varphi = (p-1) \cdot (q-1) = 4 \cdot 12 = 48$. Pour faciliter le calcul, effectuons une exponentiation modulaire rapide vu en cours en remarquant que $29 = 2^4 + 2^3 + 2^2 + 1$:

$$49^2 = (-16)^2 = 256 = -4 \pmod{65}$$

$$49^4 = (-4)^2 = 16 \pmod{65}$$

$$49^8 = 16^2 = -4 \pmod{65}$$

$$49^{16} = (-4)^2 = 16 \pmod{65}$$

On en déduit que

$$m = c^{29} = 49^{16} \cdot 49^8 \cdot 49^4 \cdot 49 = (-4) \cdot 16 \cdot 16 \cdot (-16) = (-64) \cdot 4 = 4 \pmod{65}.$$

Comme nous sommes en phase de déchiffrement, et en supposant que celui qui déchiffre a conservé p et q , on aurait aussi pu utiliser le théorème des restes chinois après avoir calculé $49^{29} \pmod{5}$ et $49^{29} \pmod{13}$ (ce qui peut aussi l'exponentiation modulaire rapide pour ces deux calculs). Par contre, cette astuce additionnelle ne fonctionne qu'en déchiffrement puisqu'il faut connaître p et q pour la mettre en œuvre.

2 Certification

1. Si Alice veut envoyer à Bob un message intègre et authentifié, est-ce qu'une certification est nécessaire? Si oui, qui va vérifier le certificat? Combien est-ce qu'Alice et Bob vont devoir générer (resp. vérifier) de signatures numériques?
2. Si Alice veut envoyer à Bob un message protégé en confidentialité, est-ce qu'une certification est nécessaire? Si oui, qui va vérifier le certificat? Combien est-ce qu'Alice et Bob vont devoir générer (resp. vérifier) de signatures numériques?

Réponse.

1. Oui, une certification est nécessaire, pour empêcher Eve de se faire passer pour Alice. Dans ce cas, c'est Bob qui va devoir vérifier le certificat qui a été généré par l'autorité de confiance pour certifier la clé publique d'Alice. Dans ce contexte, Alice va générer une seule signature numérique sur le message qu'elle veut envoyer à Bob. Elle n'aura à vérifier aucune signature. De son côté, Bob devra vérifier deux signatures, celle de l'autorité de confiance dans le certificat d'Alice, et celle d'Alice sur le message reçu. Il n'aura par contre aucune signature à générer.
2. Une certification est aussi nécessaire dans ce cas, pour empêcher Eve d'envoyer sa propre clé publique à la place de Bob pour être capable de lire les messages qu'Alice envoie. Cette fois-ci, c'est Alice qui doit vérifier

la validité du certificat de Bob, pour être sûre d'utiliser la bonne clé publique auà l'étape de chiffrement du message. Ici, Alice ne va générer aucune signature, et devra en vérifier une, celle de l'autorité de confiance dans le certificat de Bob. Bob n'aura aucune signature à générer ou à vérifier. Il n'aura qu'à déchiffrer le message reç à l'aide de sa clé privée.

3 Utilisation de RSA

Afin d'obtenir l'indistingabilité de RSA, Sébastien préconise d'utiliser la procédure suivante en chiffrement (le message est noté m), où α est une valeur fixe, et $\mathcal{G}$ et $\mathcal{H}$ sont deux fonctions de hachage.

1. Calculer $v = \mathcal{G}(\alpha)$.
2. Calculer $s = m \oplus v$.
3. Calculer $w = \mathcal{H}(s)$.
4. Calculer $t = w \oplus \alpha$.
5. Exécuter RSA sur la valeur $s||t$ pour obtenir c .
6. Envoyer c .

1. Expliquez, avec des détails, pourquoi Sébastien n'obtient pas l'indistingabilité désirée.
2. Sébastien se rend compte de son erreur et décide de choisir un α à chaque fois différent, et de renvoyer (c, α) . Est-ce mieux ?
3. Qu'aurait dû faire Sébastien ?
4. Expliquez comment doit se dérouler le déchiffrement.
5. Quelle structure reconnaissez-vous dans cette façon de procéder ?

Réponse.

1. α étant une valeur fixe, et $\mathcal{G}$ et $\mathcal{H}$ étant des fonctions de hachage (donc déterministes), les valeurs v et w sont donc tout le temps les mêmes (quel que soit le message). Comme s est fixé, si quelqu'un chiffre plusieurs fois le même message m , la valeur s sera aussi toujours la même. Ainsi, la valeur $s||t$ sera calculée de façon déterministe à partir du message : ce sera toujours le même pour un message donné. Comme RSA est aussi déterministe, cela donne donc un système de chiffrement déterministe, qui ne peut pas avoir la propriété d'indistingabilité.
2. C'est en effet mieux parce que le schéma devient ainsi probabiliste. Mais ce n'est pas encore suffisant. Dans le jeu d'indistingabilité, l'attaquant connaît la clé publique de chiffrement, et choisit deux messages m_0 et m_1 . Le challenger va alors choisir l'un des deux messages (appelons-le m_b), choisit α , puis calcule s et t , puis le chiffre RSA c de $s||t$ et envoie c et α à l'attaquant. Ce dernier connaît donc m_0 , m_1 , α et c . Il peut ainsi

choisir par exemple m_0 et faire les mêmes calculs que le challenger. S'il arrive au même chiffré, alors $b = 0$ et sinon, $b = 1$.

3. Sébastien doit effectivement choisir α mais ne doit pas surtout pas l'envoyer. Seule la valeur c doit ainsi être envoyée.
4. Lors du déchiffrement, la personne qui connaît la clé privée RSA peut, à partir de c , récupérer s et t . À partir de s et de t , il peut ensuite retrouver α et en utilisant $\mathcal{G}$ et s , obtenir m .
5. On reconnaît un schéma de Feistel.

4 Fonction de hachage et implications

Le but de l'exercice est de montrer les liens d'implication ou de non-implication parmi les propriétés des fonctions de hachage.

- Résistance à la préimage : étant donné h , on ne peut pas trouver en un temps raisonnable un x tel que $h = H(x)$.
- Résistance à la seconde préimage : étant donné x , on ne peut pas trouver en un temps raisonnable un $y \neq x$ tel que $H(y) = H(x)$.
- Résistance aux collisions : on ne peut pas trouver en un temps raisonnable de couple (x, y) tel que $H(x) = H(y)$.

Répondez aux questions suivantes.

1. Montrer que la résistance aux collisions implique la résistance à la seconde préimage.
2. Considérons la fonction $f : \mathbb{Z} \rightarrow \mathbb{Z}_n$ définie par $f(x) = x^2 \pmod{n}$ pour n un module RSA.
 - (a) Justifier que la fonction f est résistante à la préimage.
 - (b) Justifier que la fonction n'est pas résistante à la seconde préimage.
3. Considérons une fonction de hachage $g : \{0, 1\}^* \rightarrow \{0, 1\}^n$ résistante à la collision. Considérons ensuite la fonction de hachage $h : \{0, 1\}^* \rightarrow \{0, 1\}^{n+1}$ qui calcule des empreintes de longueur $n + 1$ construites de la façon suivante :

$$h(x) = \begin{cases} 1\|x & \text{si } x \text{ est de longueur } n \\ 0\|g(x) & \text{sinon.} \end{cases}$$

où $\|$ désigne l'opération de concaténation.

- (a) Montrer que cette fonction est résistante à la collision.
- (b) Montrer que cette fonction n'est pas résistante à la préimage.
4. Donner un exemple de fonction résistante aux collisions et à la seconde préimage, mais pas à la préimage. Est-ce une fonction de hachage ?
1. Raisonnons par contraposée. S'il existe une attaque pour trouver une seconde préimage, on peut facilement l'utiliser pour trouver une collision : On veut trouver de $y \neq x$ tels que $H(y) = H(x)$. Fixons un x au hasard. D'après notre hypothèse on peut facilement trouver un $y \neq x$ tel que $H(y) = H(x)$, d'où la collision.

2. Voici les deux arguments.
 - (a) Lorsque n est un module RSA, l'extraction d'une racine carrée modulo n est un problème réputé difficile, la fonction f est donc résistante à la préimage.
 - (b) Puisque $f(x) = f(-x)$, cette fonction n'est pas résistante à la seconde préimage. Ainsi, la résistance à la préimage n'entraîne pas la résistance à la seconde préimage.
3. Voici les deux raisonnements.
 - (a) Cette fonction est résistante à la collision notamment car g l'est. Si $h(x) = h(y)$ alors le premier bit ne peut être égal à 1 car sinon on aurait $h(x) = 1\|x = h(y) = 1\|y$, donc $x = y$ (ce ne serait plus une collision). Donc le premier bit est 0 et on a nécessairement $g(x) = g(y)$, ce qui veut dire que x et y forment une collision sur g . Par hypothèse, g est résistante aux collisions, donc x et y sont difficiles à trouver, et donc h est résistante aux collisions.
 - (b) Cette fonction n'est pas résistante à la préimage car il est facile de déterminer un antécédent pour la moitié des images (celles qui commencent par 1). L'antécédent de la valeur $h = 1\|x$ est juste la suite binaire x . Ainsi, la résistance à la collision n'entraîne pas la résistance à la préimage.
4. La fonction identité est résistante (infiniment) aux collisions et aux secondes préimages. Par contre, ce n'est pas une fonction de hachage car elle ne condense pas le message.

5 Mauvaise utilisation de la cryptographie

Une société Lambda veut mettre en œuvre un système cryptographique pour un nouveau produit qu'elle souhaite mettre sur le marché en Europe. Dans son cas, une partie des calculs cryptographiques sera effectuée par un composant hardware qu'elle a acheté auprès d'une société Gamma en Italie qui a une certification de l'ANSSI. Pour une autre partie, le travail a été confié à un stagiaire de Lambda pour une implémentation software. Pour la génération du secret x Diffie-Hellman, le stagiaire a fait le choix de générer un nombre aléatoire et de l'incrémenter à chaque nouvelle itération du protocole. Pour une partie du système, l'équipe cryptographique de la société Lambda a décidé de prendre AES-128 mais pour un souci de rapidité, de ne faire que 9 tours au lieu de 10. Enfin, deux modes opératoires ont été choisis et sont utilisés en parallèle sur chaque message envoyé : le mode ECB pour assurer la confidentialité et le mode CBC pour assurer l'intégrité et l'authenticité.

1. Pointer du doigt tout ce qui ne va pas dans les choix de cette société et expliquer en quelques mots ce qu'elle aurait dû faire à la place.

Correction.

1. Voici quelques éléments qui peuvent être discutés avec les étudiants.
 - Confier le développement à un unique stagiaire.
 - La génération de l'aléa x n'est pas bon car il ne faut partir d'un aléa et faire une simple incrémentation. Si l'un est corrompu, l'ensemble le sera.
 - Il faut nécessairement conserver AES en 10 tours, et ne jamais modifier un standard.
 - Le mode ECB n'est pas à utiliser pour assurer la confidentialité.
 - Le mode "Encrypt-and-MAC" n'est pas une façon idéale de procéder. Il faut lui préférer le mode "Encrypt-then-MAC"